

Reviewer Pack — Protocol-Induced Covers from Low-Cost Protocols on High-Dist...

Entry 634c56408dd3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 18:05:07 UTC

Protocol-Induced Covers from Low-Cost Protocols on High-Distortion Gadget Lifts Admit Sparse Roe Decompositions

Entry ID: 634c56408dd3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 18:05:07 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) — Geometric Complexity Theory

Field B (complexity object): Communication complexity class $CC(f \circ G^n)$

Statement:

For any MetricGadget G where the maximal Hilbertian distortion of $(X \times Y, d)$ embeddings into $(X \times Y)^n$ under $d \oplus$ grows at least as $\Omega(\log n)$, and for any deterministic communication protocol Π computing $f \circ G^n$ with cost c , the induced ProtocolPullback forms a RoeCover of scale $R_\Pi = O(\text{diam}(G))$ and multiplicity $m_\Pi \leq 2^{c - \Omega(Q(f) \cdot \log \text{asdim}_R(G))}$, assuming $\text{asdim}_R(G) \geq 1$ for $R = \text{diam}(G)$.

Rationale (proposer's reasoning):

This conjecture tests Axiom A2 (Protocol $\rightarrow$ Cover) in conjunction with the geometric obstruction posed by high embedding distortion, which limits how efficiently transcripts can cover the space without incurring coarse multiplicity. High distortion implies that Euclidean (hence protocol-induced) partitions cannot align well with the intrinsic geometry of the lift, forcing higher-than-expected multiplicity in any valid RoeCover. It leverages A3 (Asdim Amplification) indirectly by requiring $\text{asdim}_R(G)$ to be nontrivial, thereby amplifying the gap between protocol-induced covers and optimal ones. If true, it strengthens the lifting bound by tying communication cost not just to existence of low-multiplicity covers, but to their geometric realizability under distortion barriers.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5cffeb6a35907af0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Supports conjecture if multiplicity is sparse for low-cost protocols

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - coarse geometric lifting AND communication complexity AND hilbertian distortion - geometric complexity theory AND gadget lifts AND sparse roe decompositions - metric gadgets AND protocol induced covers AND logarithmic distortion growth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_random_string(length):
    return ''.join(random.choice('01') for _ in range(length))

def hamming_distance(s1, s2):
    return sum(c1 != c2 for c1, c2 in zip(s1, s2))

def generate_instances(gadget, n):
    instances = []
    for i in range(n):
        x = generate_random_string(len(list(gadget.keys())[0][0]))
        y = generate_random_string(len(list(gadget.keys())[0][1]))
        instances.append((x, y))
    return instances

def euclidean_distance(p1, p2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))

def generate_metric_gadget(k):
    gadget = {}
    for i in range(2**k):
        x = format(i, f'0{k}b')
        for j in range(2**k):
            y = format(j, f'0{k}b')
            d = hamming_distance(x, y)
            gadget[(x, y)] = d
    return gadget
```

```

def generate_protocol_pullback(gadget, protocol, n):
    pullback = {}
    for i in range(n):
        x, y = protocol[i]
        pullback[(i, x, y)] = gadget[(x, y)]
    return pullback

def compute_cover_multiplicity(pullback, n):
    cover = set()
    for i in range(n):
        for j in range(i+1, n):
            if pullback[(i, '0', '0')] + pullback[(j, '0', '0')] <= pullback[(i, '0', '0')]:
                cover.add((i, j))
    return len(cover)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    k = 5
    n = 6
    f = "Disjointness" # or "Inner Product"
    Q_f = n

    gadget = generate_metric_gadget(k)
    protocol = [(generate_random_string(k), generate_random_string(k)) for _ in range(n)]
    pullback = generate_protocol_pullback(gadget, protocol, n)

    cover_multiplicity = compute_cover_multiplicity(pullback, n)
    R_pi = max(pullback.values())
    alpha = math.log2(2)
    m_pi_bound = 2 ** (Q_f - alpha * Q_f)

    conjecture_holds = cover_multiplicity > m_pi_bound
    counterexample = "" if conjecture_holds else "multiplicity_not_sparse"

    return {
        "metric_name": "cover_multiplicity",
        "metric_value": cover_multiplicity,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"multiplicity_not_sparse\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ff16270.py", line 94, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ff16270.py", line 72, in run_trial
    cover_multiplicity = compute_cover_multiplicity(pullback, n)
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ff16270.py", line 56, in compute_cover_multiplicity
    if pullback[(i, '0', '0')] + pullback[(j, '0', '0')] <= pullback[(i, '0', '0')]:
       ~~~~~^
KeyError: (0, '0', '0')

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, and the pre-registered support condition was not unambiguously met. | next: Fix the KeyError in the test code and re-run the simulations to produce valid data.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	groq	llama-3.3-70b-versat	0	0	4329
2	novelty	groq	llama-3.3-70b-versat	0	0	4273
3	novelty	groq	llama-3.3-70b-versat	0	0	4563
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15841
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10464

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	judge	groq	llama-3.3-70b-versat	0	0	4681

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 44150 ms total latency. Provider mix: {'groq': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/634c56408dd3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/634c56408dd3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/634c56408dd3.tar.gz (if generated)